

✅ Got it — here is your **DAY 17 (clean, consolidated, no confusion)**

👉 This is **ALL changes from Day 16 → now**, merged properly into one stage.

🚀 ✅ **DAY 17 — PROJECT + DVP AWARE INTELLIGENT RETRIEVAL**

✅ 📌 **WHERE YOU WERE (DAY 16)**

- ✅ multi-PDF ingestion
 - ✅ project detection (from "Project No")
 - ✅ metadata tagging (project)
 - ✅ basic RAG working
-

✅ 🎯 **WHAT DAY 17 ADDS**

- ✅ DVP-aware retrieval (generic)
 - ✅ semantic routing (no hardcoding)
 - ✅ noise removal (tyre pressure issue FIX)
 - ✅ correct document selection
 - ✅ clean final answers
- ..
-

✅ 🧠 **CORE UPGRADE**

From:

project-aware search ✅

to:

project + document-type (DVP) aware search ✅✅

..

✅ 🏗️ **FINAL ARCHITECTURE (DAY 17)**

User Query

↓

detect_project 

↓

Retrieve project chunks 

↓

DVP semantic filtering  ← NEW

↓

Noise removal  ← NEW

↓

Clean context 

↓

Qwen 

↓

Accurate answer 

``

STEP 1 — ADD DVP TEXT (pipeline.py)

Add this function

```
def extract_dvp_text(pages, file_path):
```

```
    import os
```

```
    first_page = pages[0]["text"]
```

```
    for line in first_page.split("\n"):
```

```
        if "evaluation" in line.lower() or "test" in line.lower():
```

```
return line.strip()
```

```
return os.path.basename(file_path).replace(".pdf", "")
```

✓ Use inside process_pdf()

```
dvp_text = extract_dvp_text(pages, file_path)
```

✓ Attach to chunks

for c in chunks:

```
c["metadata"]["project"] = project_name
```

```
c["metadata"]["dvp_text"] = dvp_text
```

✓ Text chunks also

for t in text_chunks:

```
t["metadata"]["project"] = project_name
```

```
t["metadata"]["dvp_text"] = dvp_text
```

✓ ✓ STEP 2 — FILTER BY DVP (ask.py)

✓ Add function

```
def filter_by_dvp(query, results):
```

```
    query_emb = np.array(create_embedding(query)).reshape(1, -1)
```

```
    scored = []
```

```
    for r in results:
```

```
dvp_text = r["metadata"].get("dvp_text")
```

```
if not dvp_text:
```

```
    continue
```

```
dvp_emb = np.array(create_embedding(dvp_text)).reshape(1, -1)
```

```
score = cosine_similarity(query_emb, dvp_emb)[0][0]
```

```
#  REMOVE NOISE
```

```
if score < 0.5:
```

```
    continue
```

```
scored.append((score, r))
```

```
scored.sort(reverse=True, key=lambda x: x[0])
```

```
return [r for s, r in scored[:5]]
```

STEP 3 — INTEGRATE FILTER (CRITICAL)

BEFORE

```
cleaned_chunks = deduplicate_context(tool_output['results'])
```

AFTER

```
results = tool_output["results"]
```

```
filtered_results = filter_by_dvp(query, results)
```

```
cleaned_chunks = deduplicate_context(filtered_results)
```

✅✅ STEP 4 — PROJECT DETECTION (unchanged)

```
def detect_project(query):
```

```
    match = re.search(r"\b[A-Z]{1,4}\d{2,4}\b", query.upper())
```

```
    if match:
```

```
        return match.group(0)
```

```
    return None
```

✅✅ STEP 5 — FINAL FLOW (IMPORTANT)

Query:

"What is temp in thermal capacity for P201?"

↓

detect_project → P201 ✅

↓

Retrieve ALL P201 chunks ✅

↓

filter_by_dvp (semantic match) 

↓

Remove wrong chunks (tyre, config, etc.) 

↓

Qwen → correct answer 

  **WHAT THIS FIXED (VERY IMPORTANT)**

 **BEFORE**

Temperature → picked:

37 PSI  (tyre pressure)

 **NOW**

Temperature → picked:

442°C  (correct)

  **WHAT YOU BUILT**

 project-aware system

 document-aware system

 semantic routing (NO hardcoding)

 noise-free retrieval

 correct engineering answers

  **QUALITY LEVEL**

 enterprise-grade RAG 

✓ scalable across reports ✓

✓ accurate engineering QA ✓

..

✓ ✓ **FINAL ONE LINE**

Day 17 = Semantic DVP filtering + noise removal → correct document + correct answer ✓

✓ ✓ **NEXT (if you go ahead)**

DAY 18 →

→ comparison (P201 vs P202)

→ graph understanding

If you want: ✓ I'll give Day 18 in same clean template

✓ or debug your system fully

Just say 👍